原文地址

> https://towardsdatascience.com/introduction-to-weight-quantization-2494701b9c0c/
>
> https://towardsdatascience.com/4-bit-quantization-with-gptq-36b0f4f02c34/

1 权重量化简介

- 大型语言模型(LLM) 以其庞大的计算需求而闻名。通常，模型的大小是通过将参数数量（ **大小** ）乘以这些值的精度（ **数据类型** ）来计算的。但是，为了节省内存，可以通过称为量化的过程，使用较低精度的数据类型来存储权重。

  以下列举了7B和32B模型在全精度 (FP32类型) 下的模型参数大小

  7B参数模型

  1. 参数数量：7B = 7 × 10^9
  2. 每个参数的精度：FP32 = 32 bits = 4 Bytes
  3. 模型大小 = 参数数量 × 每个参数的精度 = 7 × 10^9 × 4 Bytes = 28 × 10^9 Bytes
  4. 将Bytes转换为GB：28 × 10^9 Bytes / (1024 × 1024 × 1024) ≈ 26.21 GB

  32B参数模型

  1. 参数数量：32B = 32 × 10^9
  2. 每个参数的精度：FP32 = 32 bits = 4 Bytes
  3. 模型大小 = 参数数量 × 每个参数的精度 = 32 × 10^9 × 4 Bytes = 128 × 10^9 Bytes
  4. 将Bytes转换为GB：128 × 10^9 Bytes / (1024 × 1024 × 1024) ≈ 119.21 GB

- 我们在文献中区分了两种主要的权重量化技术：

  Post-Training Quantization (PTQ)： 训练后量化，是一种简单的技术，它将已训练模型的权重转换为较低的精度，而无需重新训练。虽然 PTQ 易于实现，但它可能会导致性能下降。

  Quantization-Aware Trainin（QAT）：量化感知训练，在预训练或微调阶段加入了权重转换过程，从而提升了模型性能。然而，QAT 的计算成本较高，并且需要具有代表性的训练数据。

- 在本文中，我们重点介绍PTQ来降低参数的精度。为了获得更好的直观感受。我们将使用GPT-2模型，在一个玩具示例中分别应用简单和更复杂的技术。

1.1 浮点表示的背景

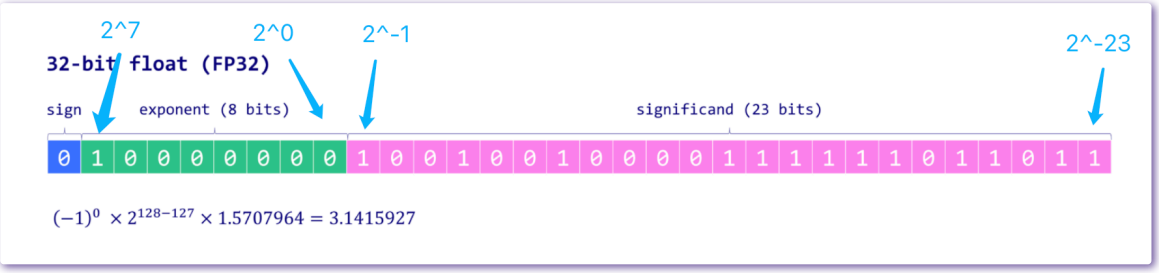
- 数据类型的选择决定了所需的计算资源数量，从而影响模型的速度和效率。在深度学习应用中，平衡精度和计算性能至关重要，因为更高的精度通常意味着更高的计算需求。
- 在众多数据类型中，浮点数因其能够高精度地表示各种值而广泛应用于深度学习。通常，浮点数使用 n 位来存储数值。这 n 位又可进一步划分为三个不同的部分：
  1. **Sign (符号)**： 符号位表示数字的正负性质。它占用一位，其中 0 表示正数，1 表示负数。
  2. **Exponent (指数)**： 指数是一段表示底数（二进制通常为 2）的幂的位。指数可以是正数或负数，从而表示非常大或非常小的值。
  3. **Significand/Mantissa (尾数/有效位)**：剩余的位用于存储有效位，也称为尾数。它代表数字的有效数字。数字的精度很大程度上取决于有效位的长度。
- 这种设计允许浮点数以不同的精度覆盖更广泛的值。其表示公式为：

$$
\begin{aligned}
&(-1)^{\text{sign}} \times \text{base}^{\text{exponent}} \times \text{significand} \\
&= (-1)^{\text{sign}} \times \text{base}^{\text{exponent}} \times (1 + \text{fraction})
\end{aligned}
\tag{1}
$$

  为了更好地理解这一点，让我们深入研究深度学习中最常用的一些数据类型：float32（FP32）、float16（FP16）和 bfloat16（BF16）

1.2 FP32

- **FP32** 使用 32 位来表示一个数字：1 位表示符号位，8 位表示指数位，其余 23 位表示尾数。虽然 FP32 提供了较高的精度，但它的缺点是计算量和内存占用较高。



  在FP32表示中，

1. sign : $\mathbf{sign} = (-1)^0 = 1$
2. exponent： $\mathbf{exponent} = 2^{2^7-127}$
3. significand:

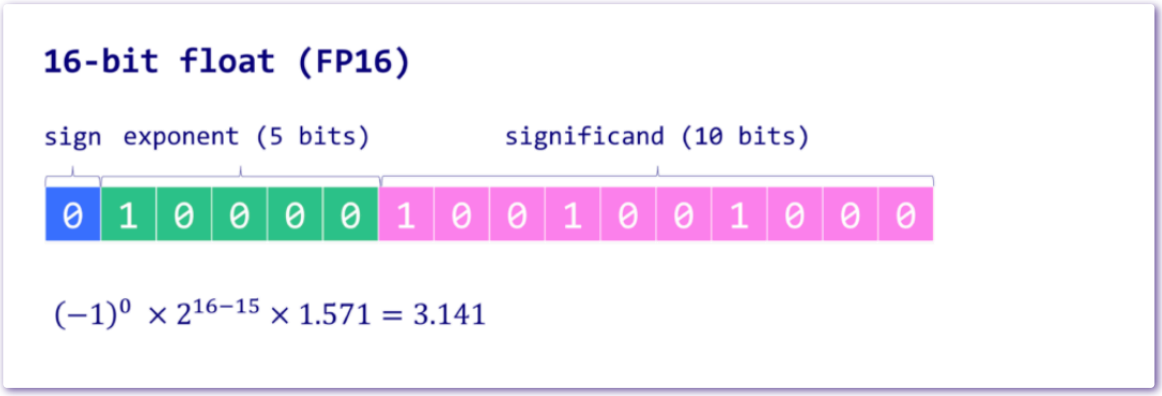
$$\begin{aligned}\mathbf{significand} &= 1 + \mathbf{fraction} \\ &= 1 + 2^{-1} + 2^{-4} + 2^{-7} + 2^{-12} + 2^{-13} + 2^{-14} + 2^{-15} + 2^{-16} + 2^{-17} + 2^{-19} + 2^{-20} + 2^{-22} + 2^{-} \\ &= 1 + 0.5707963705062866 \\ &= 1.5707963705062866\end{aligned}$$

所以上面表示的数是

$$(-1)^0 \times 2^{128-127} \times 1.5707963705062866 \tag{3}$$

1.3 FP16

- FP16** 使用 16 位来存储数字：1 位用于符号位，5 位用于指数位，10 位用于尾数。虽然这提高了内存效率并加快了计算速度，但范围和精度的降低可能会导致数值不稳定性， 从而可能影响模型精度。



在FP16表示中，

1. sign : $\mathbf{sign} = (-1)^0 = 1$
2. exponent： $\mathbf{exponent} = 2^{2^5-15}$
3. significand:

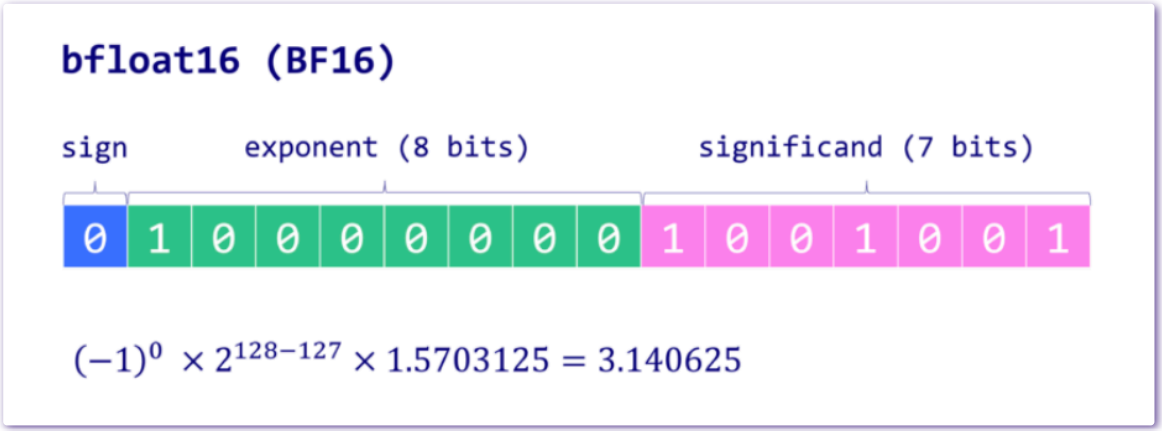
$$\begin{aligned}\mathbf{significand} &= 1 + \mathbf{fraction} \\ &= 1 + 2^{-1} + 2^{-4} + 2^{-7} \\ &= 1 + 0.5703125 \\ &= 1.5703125\end{aligned} \tag{4}$$

所以上面表示的数是

$$(-1)^0 \times 2^{16-15} \times 1.5703125 \tag{5}$$

1.4 BF16

- **BF16** 也是 16 位格式，但其中 1 位表示符号位， *8 位* 表示指数位， *7 位* 表示尾数。与 FP16 相比，BF16 扩展了可表示范围，从而降低了下溢和上溢风险。尽管由于尾数位减少导致精度降低，但 BF16 通常不会显著影响模型性能，对于深度学习任务而言，这是一种有效的折衷方案。



在BF16表示中，

1. sign : $\mathbf{sign} = (-1)^0 = 1$
2. exponent： $\mathbf{exponent} = 2^{2^7-128}$
3. significand:

$$\begin{aligned}\mathbf{significand} &= 1 + \mathbf{fraction} \\ &= 1 + 2^{-1} + 2^{-4} + 2^{-7} \\ &= 1 + 0.5703125 \\ &= 1.5703125\end{aligned} \tag{6}$$

所以上面表示的数是

$$(-1)^0 \times 2^{16-15} \times 1.5703125 \tag{7}$$

- 在机器学习术语中，FP32 通常被称为“全精度”（4 字节），而 BF16 和 FP16 被称为“半精度”（2 字节）。但我们能否做得更好，用一个字节来存储权重？答案是 INT8 数据类型，它由 8 位表示形式组成，能够存储 2^8 = 256 个不同的值。在下一节中，我们将了解如何将 FP32 权重转换为 INT8 格式。

1.5 简单的8位量化

- 在本节中，我们将实现两种量化技术：一种是采用**绝对最大值 (absmax) 量化**的对称量化技术，另一种是采用**零点量化的非对称量化技术** 。这两种量化技术的目标都是将 FP32 张量 **X** （原始权重）映射到 INT8 张量 **X_quant** （量化权重）。

1.6 绝对最大值量化

- 对称量化技术
- 使用**绝对最大值量化**时，原始数字会除以张量的绝对最大值，再乘以缩放因子 (127)，以将输入映射到 [-127, 127] 范围内。为了检索原始 FP16 值，需要将 INT8 数字除以量化因子，并承认由于舍入而导致的一些精度损失。

$$\mathbf{X}_{\text{quant}} = \mathbf{round}(\frac{\mathbf{X}}{\mathbf{max}|\mathbf{X}|} \cdot 127) \tag{8}$$

$$\mathbf{X}_{\text{dequant}} = \frac{\mathbf{X}_{\text{quant}}}{127} \cdot \mathbf{max}|\mathbf{X}| \tag{9}$$

例如，假设绝对最大值为 3.2。权重 0.1 将被量化为

$$\begin{aligned}\mathbf{X}_{\text{quant}} &= \mathbf{round}(0.1/3.2 \times 127) \\ &= \mathbf{round}(3.96875) \\ &= 4\end{aligned} \tag{10}$$

。如果我们要对其进行反量化，则会得到

$$\begin{aligned}\mathbf{X}_{\text{dequant}} &= 4/127 \times 3.2 \\ &= 0.1008\end{aligned} \tag{11}$$

，这意味着误差为 0.008。以下是相应的 Python 实现：

```python
import torch

def absmax_quantize(X):
    # Calculate scale
    scale = 127 / torch.max(torch.abs(X))

    # Quantize
    X_quant = (scale * X).round()

    # Dequantize
    X_dequant = X_quant / scale

    return X_quant.to(torch.int8), X_dequant
```

1.7 零点量化

- 非对称量化技术

- 通过**零点量化**，我们可以考虑非对称输入分布，这在考虑 ReLU 函数的输出（例如，仅考虑正值）时非常有用。首先，将输入值按值的总范围 (255) 除以最大值和最小值之差进行缩放。然后，将该分布按零点偏移，将其映射到 [-128, 127] 范围内（注意与 absmax 相比的额外值）。首先，我们计算缩放因子和零点值：

$$\mathbf{sacle} = \frac{255}{\mathbf{max}(X) - \mathbf{min}(X)} \tag{12}$$

$$\mathbf{zeropoint} = -\mathbf{round}\left[\mathbf{scale} \cdot \mathbf{min}(\mathbf{X})\right] - 128 \tag{13}$$

然后，我们可以使用这些变量来量化或反量化我们的权重：

$$\mathbf{X}_{\mathbf{quant}} = \mathbf{round}(\mathbf{scale} \cdot \mathbf{X} + \mathbf{zeropoint}) \tag{14}$$

$$\mathbf{X}_{\mathbf{dequant}} = \frac{\mathbf{X}_{\mathbf{quant}} - \mathbf{zeropoint}}{\mathbf{scale}} \tag{15}$$

举个例子：最大值为 3.2，最小值为 -3.0。我们可以计算出比例为

$$\mathbf{scale} = 255/(3.2 + 3.0) = 41.13 \tag{16}$$

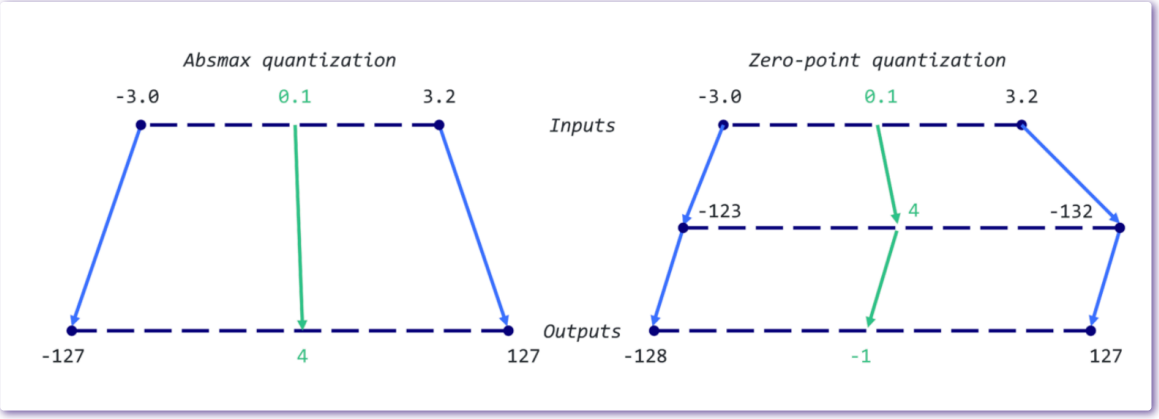
零点为

$$\begin{aligned} \mathbf{zeropoint} &= -round(41.13\times - 3.0)–128 \\ &= 123 - 128 \\ &= -5 \end{aligned} \tag{17}$$

因此之前的权重 0.1 将被量化为

$$\begin{aligned} \mathbf{X}_{\mathbf{quant}} &= \mathbf{round}(41.13\times 0.1 + (-5)) \\ &= -1 \end{aligned} \tag{18}$$

这与之前使用 absmax 获得的值（4 vs. -1）截然不同。



- 零点量化的python实现也非常简单

```python
import torch

def zeropoint_quantize(X):
    # Calculate value range (denominator)
    x_range = torch.max(X) - torch.min(X)
    x_range = 1 if x_range == 0 else x_range

    # Calculate scale
    scale = 255 / x_range

    # Shift by zero-point
    zeropoint = (-scale * torch.min(X) - 128).round()

    # Scale and round the inputs
    X_quant = torch.clip((X * scale + zeropoint).round(), -128, 127)

    # Dequantize
    X_dequant = (X_quant - zeropoint) / scale

    return X_quant.to(torch.int8), X_dequant
```

- 借助 `transformers` 库，我们可以在真实模型上使用这两个函数，而不必依赖完整的玩具示例。
- 我们首先加载 GPT-2 的模型和分词器。这是一个非常小的模型，我们可能不想对其进行量化，但对于本教程来说已经足够了。首先，我们需要观察模型的大小，以便稍后进行比较，并评估 8 位量化带来的**内存节省**。

安装下面的库

```
pip install'bitsandbytes>=0.39.0'
pip install accelerate
pip install transformers
```

```
from transformers import AutoModelForCausalLM, AutoTokenizer
import torch
torch.manual_seed(0)

# Set device to CPU for now
device = 'cpu'

# Load model and tokenizer
model_id = 'gpt2'
model = AutoModelForCausalLM.from_pretrained(model_id).to(device)
tokenizer = AutoTokenizer.from_pretrained(model_id)

# Print model size
print(f"Model size: {model.get_memory_footprint():,} bytes")


"""输出如下"""
Model size: 510,342,192 bytes
```

GPT-2 模型在 FP32 下的大小约为 $510,342,192/1024/1024 \approx 487MB$。下一步是使用零点量化和绝对最大量化来量化权重。在以下示例中，我们将这些技术应用于 GPT-2 的第一个注意力层，以查看结果。

```
# Extract weights of the first layer
weights = model.transformer.h[0].attn.c_attn.weight.data
print("Original weights:")
print(weights)

# Quantize layer using absmax quantization
weights_abs_quant, _ = absmax_quantize(weights)
print("nAbsmax quantized weights:")
print(weights_abs_quant)

# Quantize layer using absmax quantization
weights_zp_quant, _ = zeropoint_quantize(weights)
print("nZero-point quantized weights:")
print(weights_zp_quant)
```

输出如下

```
Original weights:
tensor([[-0.4738, -0.2614, -0.0978,  ...,  0.0513, -0.0584,  0.0250],
        [ 0.0874,  0.1473,  0.2387,  ..., -0.0525, -0.0113, -0.0156],
        [ 0.0039,  0.0695,  0.3668,  ...,  0.1143,  0.0363, -0.0318],
        ...,
        [-0.2592, -0.0164,  0.1991,  ...,  0.0095, -0.0516,  0.0319],
        [ 0.1517,  0.2170,  0.1043,  ...,  0.0293, -0.0429, -0.0475],
        [-0.4100, -0.1924, -0.2400,  ..., -0.0046,  0.0070,  0.0198]],
       device='cuda:0',
       dtype=torch.float32)

Absmax quantized weights:
tensor([[-21, -12,  -4,  ...,   2,  -3,   1],
        [  4,   7,  11,  ...,  -2,  -1,  -1],
        [  0,   3,  16,  ...,   5,   2,  -1],
        ...,
        [-12,  -1,   9,  ...,   0,  -2,   1],
        [  7,  10,   5,  ...,   1,  -2,  -2],
        [-18,  -9, -11,  ...,   0,   0,   1]], device='cuda:0',
       dtype=torch.int8)

Zero-point quantized weights:
tensor([[-20, -11,  -3,  ...,   3,  -2,   2],
        [  5,   8,  12,  ...,  -1,   0,   0],
        [  1,   4,  18,  ...,   6,   3,   0],
        ...,
        [-11,   0,  10,  ...,   1,  -1,   2],
        [  8,  11,   6,  ...,   2,  -1,  -1],
```

```
         [-18,  -8, -10,  ...,   1,   1,   2]], device='cuda:0',
        dtype=torch.int8)
```

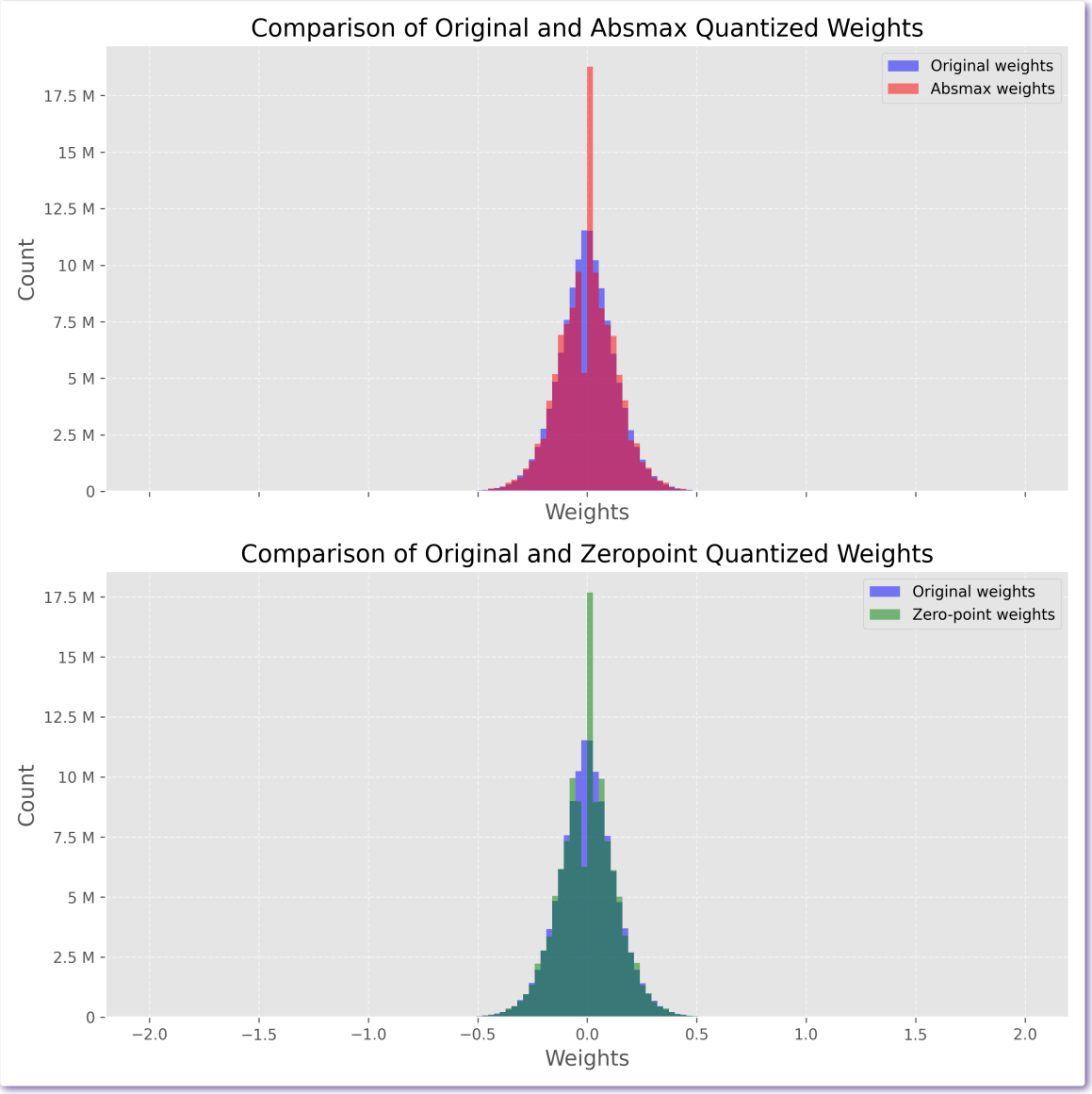
原始值 (FP32) 和量化值 (INT8) 之间的差异很明显，但绝对最大权重和零点权重之间的差异则更为微妙。在本例中，输入看起来偏移了 -1。这表明该层的权重分布相当对称。

- 我们可以通过量化 GPT-2 中的每一层（线性层、注意力层等）来比较这些技术，并创建两个新模型： `model_abs` 和 `model_zp` 。准确地说，我们实际上会用去 ***量化*** 的权重替换原始权重。这样做有两个好处：1/ 它允许我们比较权重的分布（相同尺度）；2/ 实际运行模型
- 确实，PyTorch 默认不允许进行 INT8 矩阵乘法。在实际场景中，我们会对它们进行反量化以运行模型（例如在 FP16 中），但将它们存储为 INT8。在下一节中，我们将使用 `[bitsandbytes]`

```python
import numpy as np
from copy import deepcopy

# Store original weights
weights = [param.data.clone() for param in model.parameters()]

# Create model to quantize
model_abs = deepcopy(model)

# Quantize all model weights
weights_abs = []
for param in model_abs.parameters():
    _, dequantized = absmax_quantize(param.data)
    param.data = dequantized
    weights_abs.append(dequantized)

# Create model to quantize
model_zp = deepcopy(model)

# Quantize all model weights
weights_zp = []
for param in model_zp.parameters():
    _, dequantized = zeropoint_quantize(param.data)
    param.data = dequantized
    weights_zp.append(dequantized)
```

- 现在我们的模型已经量化，我们想检查这个过程的影响。直观地说，我们要确保量化后的权重**接近原始权重** 。一个直观的检查方法是绘制反量化后权重和原始权重的分布。如果量化是有损的，它会极大地改变权重分布。

  下图显示了两者的比较，其中蓝色直方图表示原始 (FP32) 权重，红色直方图表示去量化 (来自 INT8) 权重。请注意，我们仅显示 -2 到 2 之间的此图，因为存在绝对值非常高的异常值（稍后会详细介绍）。



- 两幅图非常相似，在 0 附近有一个令人惊讶的尖峰。这个尖峰表明我们的量化过程损耗很大，因为逆转过程不会输出原始值。对于 absmax 模型尤其如此，它在 0 附近既显示出较低的谷值，也显示出较高的尖峰。

  让我们比较一下原始模型和量化模型的性能。为此，我们定义了一个 `generate_text()` 函数，通过 **top-k 采样** 生成 50 个 token。

```python
def generate_text(model, input_text, max_length=50):
    input_ids = tokenizer.encode(input_text, return_tensors='pt').to(device)
    output = model.generate(inputs=input_ids,
                            max_length=max_length,
                            do_sample=True,
                            top_k=30,
                            pad_token_id=tokenizer.eos_token_id,
                            attention_mask=input_ids.new_ones(input_ids.shape))
    return tokenizer.decode(output[0], skip_special_tokens=True)

# Generate text with original and quantized models
original_text = generate_text(model, "I have a dream")
absmax_text   = generate_text(model_abs, "I have a dream")
zp_text       = generate_text(model_zp, "I have a dream")

print(f"Original model:\n{original_text}")
print("-" * 50)
print(f"Absmax model:\n{absmax_text}")
print("-" * 50)
print(f"Zeropoint model:\n{zp_text}")
```

输出如下

```
Original model:
```

```
I have a dream. I don't know what will come of it, but I am going to have to look for something that will be right. I haven't thought about it for a long time, but I have to try to get that thing
--------------------------------------------------
Absmax model:
I have a dream job.

If you can talk about your work and what you believe, then come on! I'm gonna love you!

Letters in support of this article were also accepted on Facebook
--------------------------------------------------
Zeropoint model:
I have a dream!"

This was followed by "I have a dream!" "I dream!"

"My dream."

"My dream!" "I dream!"

In the first week of the holiday, the two-
```

- 我们不必费力去判断某个输出是否比其他输出更有意义，而是可以通过计算每个输出的**困惑度 (perplexity)** 来量化它。困惑度是评估语言模型的常用指标，它衡量模型在预测序列中下一个标记时的不确定性。在这种比较中，我们通常假设分数越低，模型越好。实际上，困惑度高的句子也可能是正确的。

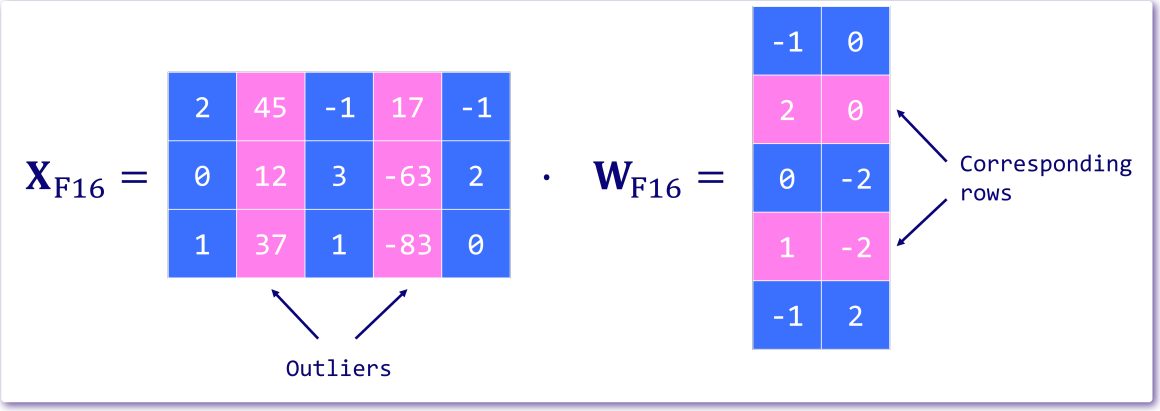
  我们使用最小函数来实现它，因为我们的句子很短，所以它不需要考虑上下文窗口的长度之类的细节。

```python
def calculate_perplexity(model, text):
    # Encode the text
    encodings = tokenizer(text, return_tensors='pt').to(device)

    # Define input_ids and target_ids
    input_ids = encodings.input_ids
    target_ids = input_ids.clone()

    with torch.no_grad():
        outputs = model(input_ids, labels=target_ids)

    # Loss calculation
    neg_log_likelihood = outputs.loss

    # Perplexity calculation
    ppl = torch.exp(neg_log_likelihood)

    return ppl

ppl     = calculate_perplexity(model, original_text)
ppl_abs = calculate_perplexity(model_abs, absmax_text)
ppl_zp  = calculate_perplexity(model_zp, absmax_text)

print(f"Original perplexity:  {ppl.item():.2f}")
print(f"Absmax perplexity:    {ppl_abs.item():.2f}")
print(f"Zeropoint perplexity: {ppl_zp.item():.2f}")


"""输出如下"""
Original perplexity:  7.94
Absmax perplexity:    18.97
Zeropoint perplexity: 21.97
```

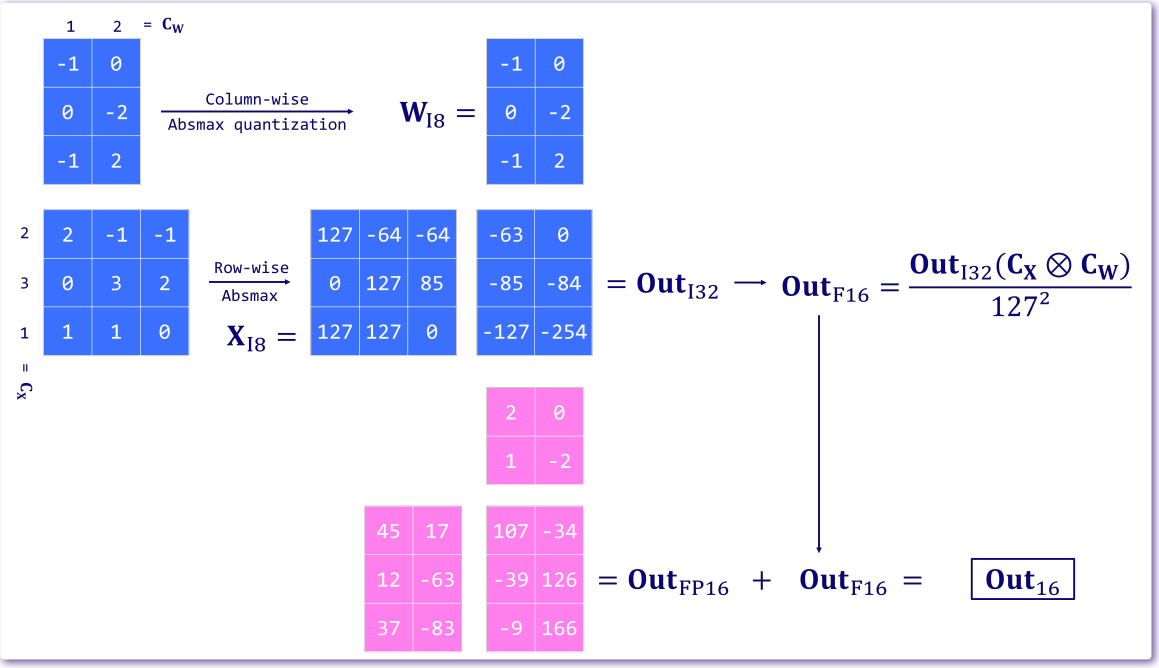
- 我们发现原始模型的困惑度**略低于**其他两个模型。单次实验的可靠性不高，但我们可以重复多次此过程，以查看每个模型之间的差异。理论上，零点量化应该比绝对最大量化略好，但计算成本也更高。
- 在此示例中，我们将量化技术应用于整个层（每个张量）。但是，我们可以将其应用于不同的粒度级别：从整个模型到单个值。一次性量化整个模型会严重降低性能，而量化单个值会产生巨大的开销。在实践中，我们通常更喜欢**向量量化**，它考虑了同一张量内行和列值的变化。
- 然而，即使是向量量化也无法解决异常特征的问题。异常特征是指当模型达到一定规模（>6.7B 个参数）时，所有 Transformer 层中都会出现的极值（负值或正值）。这是一个问题，因为单个异常值就会降低所有其他值的精度。但丢弃这些异常特征并非明智之举，因为这会**极大地降低**模型的性能。

1.8 使用 LLM.int8() 进行 8 位量化

- Dettmers 等人 (2022) 提出的 LLM.int8() 函数解决了异常值问题。它基于向量 (absmax) 量化方案，并引入了混合精度量化。这意味着异常值特征将以 FP16 格式处理以保持其精度，而其他值则以 INT8 格式处理。由于异常值约占所有值的 0.1%，这有效地将 LLM 的内存占用减少了近 2 倍。



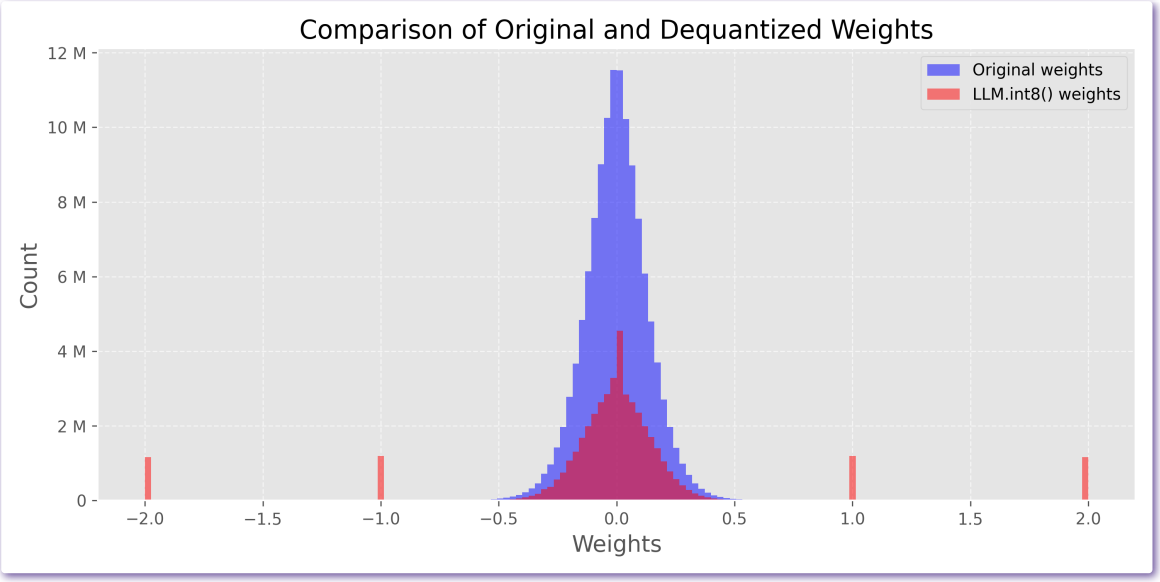
- LLM.int8() 通过三个关键步骤进行矩阵乘法计算：
    1. 使用自定义阈值从输入隐藏状态 X 中提取包含异常特征的列。
    2. 使用 FP16 对异常值进行矩阵乘法，使用 INT8 对非异常值进行矩阵乘法，并进行矢量量化（对于隐藏状态 X 则按行进行，对于权重矩阵 W 则按列进行）。
    3. 对非异常结果（INT8 到 FP16）进行去量化，并将其添加到异常结果中以获得 FP16 中的完整结果。



- 这种方法是必要的，因为 8 位精度有限，在量化大值向量时可能导致严重误差。这些误差在多层传播时也容易被放大。
- 由于 `bitsandbytes` 库已集成到 Hugging Face 生态系统中，我们可以轻松使用这项技术。我们只需在加载模型时指定 `load_in_8bit=True` （它也需要 GPU）。

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

model_int8 = AutoModelForCausalLM.from_pretrained(model_id,
                                                  device_map='auto',
                                                  load_in_8bit=True,
                                                  )
print(f"Model size: {model_int8.get_memory_footprint():,} bytes")


"""输出如下"""
Model size: 176,527,896 bytes
```

$176,527,896 bytes/1024/1024 \approx 168MB$ 加上这行代码后，模型体积几乎缩小了三倍（168MB vs. 487MB）。我们甚至可以像之前一样比较原始权重和量化权重的分布：



在这种情况下，我们看到在 -2、-1、0、1、2 等附近的峰值。这些值对应于以 INT8 格式存储的参数（非异常值）。您可以使用 `model_int8.parameters()` 打印模型的权重来验证这一点。

- 我们还可以用这个量化模型生成文本，并将其与原始模型进行比较

```
# Generate text with quantized model
text_int8 = generate_text(model_int8, "I have a dream")

print(f"Original model:n{original_text}")
print("-" * 50)
print(f"LLM.int8() model:n{text_int8}")
```

输出

```
Original model:
I have a dream. I don't know what will come of it, but I am going to have to look for something that will be right. I haven't thought about it for a long time, but I have to try to get that thing
--------------------------------------------------
LLM.int8() model:
I have a dream, I want to start writing." (This was when I was 6 years old). I have a dream, I want to start writing.

After that I started college (I was a writer for three years). I started
```

很难判断什么是最佳输出，但我们可以依靠困惑度指标来给我们一个（近似的）答案。

```
print(f"Perplexity (original):   {ppl.item():.2f}")

ppl = calculate_perplexity(model_int8, text_int8)
print(f"Perplexity (LLM.int8()): {ppl.item():.2f}")

"""输出"""
Perplexity (original):   9.42
Perplexity (LLM.int8()): 11.37
```

在这种情况下，量化模型的困惑度一般解禁原始模型。这显示出这种量化技术非常具有竞争力。事实上，LLM.int8() 的作者表明，性能下降非常低，可以忽略不计 (<1%)。然而，它在计算方面有额外的成本：对于大型模型，LLM.int8() 的速度大约会慢 20%。

1.9 conclusion

- 本文概述了最流行的权重量化技术。我们首先了解了浮点表示，然后介绍了两种 8 位量化技术： **绝对最大 (absmax)** 和**零点量化** 。然而，它们的局限性，尤其是在处理异常值方面，促成了 **LLM.int8()** 的诞生，这种技术也能保持模型的性能。这种方法突显了权重量化领域的进展，揭示了正确处理异常值的重要性。

2 使用GPTQ进行4位量化

-
- 权重量化领域的最新进展使我们能够在消费级硬件上运行大规模大型语言模型，例如在 RTX 3090 GPU 上运行 LLaMA-30B 模型。这得益于 **GPTQ** 、 **GGML** 和 **NF4** 等性能损失极小的新型 4 位量化技术。
- 在 **上一篇文章** 中，我们介绍了简单的 8 位量化技术和优秀的 LLM.int8() 函数。在本文中，我们将探索流行的 **GPTQ 算法** ，了解其工作原理，并使用 **AutoGPTQ** 库实现它。

2.1 Optimal Brain Quantization

- 首先介绍一下我们要解决的问题。对于网络中的每一层ℓ，我们希望找到*原始权重* $\mathbf{W}_\ell$ 的量化版本 $\hat{\mathbf{W}}_\ell$ 。这被称为**逐层压缩问题** 。更具体地说， 为了最大限度地减少性能下降，我们希望这些新权重的输出（ $\hat{\mathbf{W}}_\rho\mathbf{X}_\rho$ ）尽可能接近原始权重（ $\mathbf{W}_\rho\mathbf{X}_\rho$ ） 。换句话说，我们希望找到：

$$\arg\min_{\widehat{\mathbf{W}}_\ell}\left\|\mathbf{W}_\ell\mathbf{X}_\ell-\widehat{\mathbf{W}}_\ell\mathbf{X}_\ell\right\|_2^2 \tag{19}$$

  已经提出了不同的方法来解决这个问题，但我们感兴趣的是 **Optimal Brain Quantization** (OBQ) 框架。
- 该方法的灵感来源于一种**剪枝技术** ，该技术可以从完全训练的密集神经网络（Optimal Brain Surgeon）中谨慎地移除权重。它使用了一种近似技术，并提供了明确的公式，用于计算待移除的最佳单个权重 w_q 和最佳更新 δ_F ，以调整剩余的非量化权重集合 F 来弥补移除的影响：

$$\begin{aligned} w_q &= \arg\min_{w_q}\frac{\left(\operatorname{quant}\left(w_q\right)-w_q\right)^2}{\left[\mathbf{H}_F^{-1}\right]_{qq}}, \\ \delta_F &= -\frac{w_q-\operatorname{quant}\left(w_q\right)}{\left[\mathbf{H}_F^{-1}\right]_{qq}}\cdot\left(\mathbf{H}_F^{-1}\right)_{:,q}. \end{aligned} \tag{20}$$

  其中，quant(w) 是量化给出的权重舍入， $\mathbf{H}_F$ 是 Hessian。

  使用 OBQ，我们可以先量化最容易量化的权重，然后调整所有剩余的未量化权重来**补偿精度损失** 。之后，我们选择下一个要量化的权重，依此类推。
- 这种方法的一个潜在问题是，当存在异常权重时，这会导致较高的**量化误差** 。通常，这些异常值会最后量化，因为此时剩余的未量化权重很少，无法进行调整以补偿较大的误差。如果某些权重通过中间更新被进一步推到网格之外，这种影响可能会加剧。我们采用了一种简单的启发式方法来防止这种情况：异常值一出现就立即量化。
- 这个过程计算量可能很大，尤其是对于 LLM 来说。为了解决这个问题，OBQ 方法使用了一个技巧，避免每次简化权重时都重新进行整个计算。在量化权重之后，它会通过**移除与该权重相关的行和列** （使用高斯消元法）来调整用于计算的矩阵（Hessian）：

$$\mathbf{H}_{-q}^{-1}=\left(\mathbf{H}^{-1}-\frac{1}{\left[\mathbf{H}^{-1}\right]_{qq}}\mathbf{H}_{:,q}^{-1}\mathbf{H}_{q,:}^{-1}\right)_{-p}. \tag{21}$$

  该方法还采用向量化来一次性处理权重矩阵的多行。尽管 OBQ 效率很高，但随着权重矩阵规模的增加，其计算时间会显著增加。这种立方增长使得 OBQ 难以应用于包含数十亿个参数的大型模型

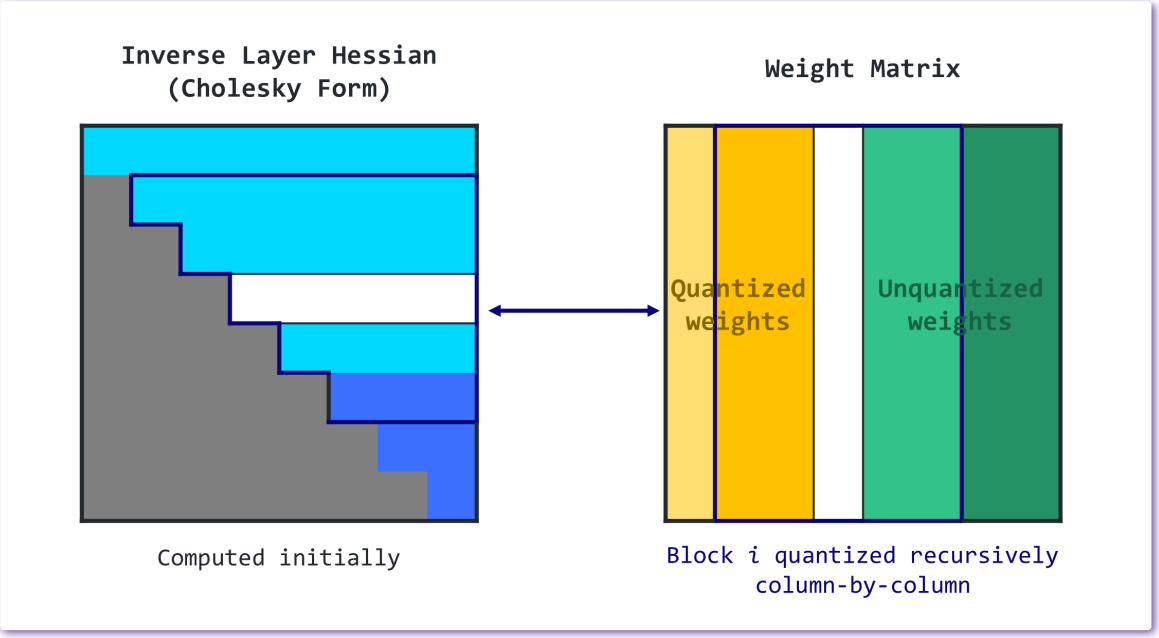
2.2 GPTQ 算法

- **GPTQ 算法** 由 Frantar 等人 (2023) 提出，它从 OBQ 方法中汲取灵感，但进行了重大改进，可以扩展到（非常）大型的语言模型。

2.2.1 步骤 1：任意顺序洞察

- OBQ 方法会选择权重（模型中的参数）按特定顺序进行量化，该顺序由**增加的额外误差最小**决定。然而，GPTQ 观察到，对于大型模型，以任何固定顺序量化权重都能获得同样好的效果。这是因为，即使某些权重单独使用时可能会引入更多误差，它们也会在量化过程的后期进行量化，因为此时剩下的其他可能增加误差的权重很少。因此，顺序并不像我们想象的那么重要

  基于这一洞见，GPTQ 旨在以**相同的顺序量化矩阵所有行**的所有权重。这使得处理速度更快，因为某些计算只需对每列进行一次，而不是对每个权重进行一次。



2.2.2 步骤 2：延迟批量更新

- 这种方案速度不快，因为它需要更新一个**庞大的矩阵**，而每个条目的计算量却很少。这类操作无法充分利用 GPU 的计算能力，并且会因内存限制（内存吞吐量瓶颈）而减慢速度。
- 为了解决这个问题，GPTQ 引入了“惰性批量”更新。事实证明，给定列的最终舍入决策仅受对该列执行的更新影响，而不会受后续列的影响。因此，GPTQ 可以**一次将算法应用于一批列**（例如 128 列），仅更新这些列及其对应的矩阵块。在完全处理完一个块后，该算法将对整个矩阵执行全局更新。

$$
\begin{aligned}
\delta_F &= -\left(\mathbf{w}_Q - \operatorname{quant}\left(\mathbf{w}_Q\right)\right)\left(\left[\mathbf{H}_F^{-1}\right]_{QQ}\right)^{-1}\left(\mathbf{H}_F^{-1}\right)_{:,Q}, \\
\mathbf{H}_{-Q}^{-1} &= \left(\mathbf{H}^{-1} - \mathbf{H}_{:,Q}^{-1}\left(\left[\mathbf{H}_F^{-1}\right]_{QQ}\right)^{-1}\mathbf{H}_{Q,:}^{-1}\right)_{-Q}.
\end{aligned}
\tag{22}
$$

2.2.3 步骤 3：Cholesky 重新表述

- 然而，还有一个问题需要解决。当算法扩展到非常大的模型时，数值不准确性可能会成为一个问题。具体来说，重复应用某个操作可能会**累积数值误差**。
- 为了解决这个问题，GPTQ 使用了 **Cholesky 分解**，这是一种用于解决某些数学问题的数值稳定方法。它涉及使用 Cholesky 方法从矩阵中预先计算一些所需信息。这种方法与轻微的“衰减”（在矩阵的对角线元素上添加一个小常数）相结合，有助于算法避免数值问题

  完整的算法可以概括为几个步骤：

  1. GPTQ 算法首先对 Hessian 逆进行 Cholesky 分解（该矩阵有助于决定如何调整权重）
  2. 然后它循环运行，一次处理一批列。
  3. 对于批次中的每一列，它量化权重，计算误差，并相应地更新块中的权重。
  4. 处理批次后，它会根据块的错误更新所有剩余的权重。
- GPTQ 算法已在各种语言生成任务上进行了测试。我们将其与其他量化方法进行了比较，例如将所有权重四舍五入到最接近的量化值 (RTN)。GPTQ 与 BLOOM（176B 参数）和 OPT（175B 参数）模型系列配合使用，并使用**单个 NVIDIA A100 GPU** 对模型进行量化。
-

2.3 使用 AutoGPTQ 量化LLM

- 这里注意啊，AutoGPTQ已经停止开发了（2024年2月之后就没有版本了）
- GPTQ 非常流行，它用于创建能够在 GPU 上高效运行的 4 位精度模型。您可以在 Hugging Face Hub 上找到许多示例，尤其是来自 **TheBloke** 的示例。如果您正在寻找一种对 CPU 更友好的方法，**GGML** 目前是您的最佳选择。最后，带有 `bitsandbytes` 的 `transformers` 库允许您在加载模型时使用 `load_in_4bit=true` 参数对其进行量化，这需要下载完整模型并将其存储在 RAM 中。
- 让我们使用 AutoGPTQ 库实现 GPTQ 算法，并量化 GPT-2 模型。这需要 GPU，但 Google Colab 上的免费 T4 即可满足需求。首先，加载库并定义要量化的模型（在本例中为 GPT-2）。

```
pip install auto-gptq
pip install transformers
```

```python
import random

from auto_gptq import AutoGPTQForCausalLM, BaseQuantizeConfig
from datasets import load_dataset
import torch
from transformers import AutoTokenizer

# Define base model and output directory
model_id = "gpt2"
out_dir = model_id + "-GPTQ"
```

- 现在我们要加载模型和分词器。分词器使用 `transformers` 库中经典的 `AutoTokenizer` 类加载。另一方面，我们需要传递一个特定的配置（ `BaseQuantizeConfig` ）来加载模型。

  在此配置中，我们可以指定要量化的位数（此处为 `bits=4` ）和组大小（惰性批次的大小）。请注意，此组大小是可选的：我们也可以对整个权重矩阵使用**一组参数** 。在实践中，这些组通常能够以非常低的成本提高量化质量（尤其是在 `group_size=1024` 情况下）。damp_percent `damp_percent` 用于辅助 Cholesky 重构，不应更改。

  最后， `desc_act` （也称为 act order）是一个比较棘手的参数。它允许你**根据激活函数的递减顺序来处理行** ，这意味着最重要或影响最大的行（由采样的输入和输出决定）会被优先处理。此方法旨在将大部分量化误差（量化过程中不可避免地引入）放在不太重要的权重上。这种方法通过确保以更高的精度处理最重要的权重，提高了量化过程的整体准确性。然而，当与组大小一起使用时， `desc_act` 可能会导致性能下降，因为需要频繁重新加载量化参数。因此，我们在这里不会使用它（不过，将来可能会修复）。

```python
# Load quantize config, model and tokenizer
quantize_config = BaseQuantizeConfig(
    bits=4,
    group_size=128,
    damp_percent=0.01,
    desc_act=False,
)
model = AutoGPTQForCausalLM.from_pretrained(model_id, quantize_config)
tokenizer = AutoTokenizer.from_pretrained(model_id)
```

  量化过程**高度依赖样本来**评估和提升量化质量。样本提供了一种比较原始模型和新量化模型输出的方法。样本数量越多，就越有可能进行更准确、更有效的比较，从而提高量化质量。

- 在本文中，我们利用 **C4（Colossal Clean Crawled Corpus，巨型干净爬取语料库）数据** 集生成样本。C4 数据集是一个从 Common Crawl 项目收集的大规模多语言网络文本集合。该数据集经过清理和预处理，专门用于训练大规模语言模型，是此类任务的理想资源。WikiText 数据集也是另一个热门选择。

  在下面的代码块中，我们从 C4 数据集中加载 1024 个样本，对其进行标记并格式化。

```python
# Load data and tokenize examples
n_samples = 1024
data = load_dataset("allenai/c4", data_files="en/c4-train.00001-of-01024.json.gz",
split=f"train[:{n_samples*5}]")
tokenized_data = tokenizer("nn".join(data['text']), return_tensors='pt')

# Format tokenized examples
examples_ids = []
for _ in range(n_samples):
    i = random.randint(0, tokenized_data.input_ids.shape[1] - tokenizer.model_max_length - 1)
    j = i + tokenizer.model_max_length
    input_ids = tokenized_data.input_ids[:, i:j]
    attention_mask = torch.ones_like(input_ids)
    examples_ids.append({'input_ids': input_ids, 'attention_mask': attention_mask})
```

  现在数据集已准备就绪，我们可以以批次大小 1 开始量化过程。我们还可以选择使用 **OpenAI Triton** （CUDA 的替代方案）与 GPU 通信。完成后，我们将标记器和模型保存为 safetensors 格式。

```python
# Quantize with GPTQ
model.quantize(
    examples_ids,
    batch_size=1,
    use_triton=True,
)

# Save model and tokenizer
model.save_quantized(out_dir, use_safetensors=True)
tokenizer.save_pretrained(out_dir)
```

- 按照惯例，可以使用 `AutoGPTQForCausalLM` 和 `AutoTokenizer` 类从输出目录加载模型和标记器

```python
device = "cuda:0" if torch.cuda.is_available() else "cpu"

# Reload model and tokenizer
model = AutoGPTQForCausalLM.from_quantized(
    out_dir,
    device=device,
    use_triton=True,
    use_safetensors=True,
)
tokenizer = AutoTokenizer.from_pretrained(out_dir)
```

让我们检查一下模型是否正常工作。AutoGPTQ 模型（大部分情况下）可以作为普通的 `transformers` 模型运行，这使得它与推理流程兼容，如下例所示：

```python
from transformers import pipeline

generator = pipeline('text-generation', model=model, tokenizer=tokenizer)
result = generator("I have a dream", do_sample=True, max_length=50)[0]['generated_text']
print(result)

"""输出如下"""
I have a dream," she told CNN last week. "I have this dream of helping my mother find her own. But, to tell that for the first time, now that I'm seeing my mother now, just knowing how wonderful it is that

```

- 我们成功地从量化后的 GPT-2 模型中获得了令人信服的完备性。更深入的评估需要**测量量化模型与原始模型的困惑度** 。不过，这部分内容不在本文的讨论范围内。

2.4 结论

- 在本文中，我们介绍了 GPTQ 算法，这是一种在消费级硬件上运行 LLM 的先进量化技术。我们展示了它如何基于改进的 OBS 技术（该技术具有任意阶洞察、惰性批量更新和 Cholesky 重构功能）解决逐层压缩问题。这种新颖的方法**显著降低了内存和计算需求** ，使 LLM 能够被更广泛的用户所接受。
- 此外，我们在免费的 T4 GPU 上**量化了我们自己的 LLM 模型** ，并运行它来生成文本。您可以在 Hugging Face Hub 上推送您自己的 GPTQ 4 位量化模型版本。正如简介中提到的，GPTQ 并非唯一的 4 位量化算法： **GGML** 和 **NF4** 也是优秀的替代方案，只是适用范围略有不同。我鼓励您进一步了解它们并尝试一下！